

BAN404 cheatsheet - modellkort

Hver metode delt i forklaring, manuell kode/formel, pakkekode og eksamenstips

2026-05-18

0. Felles oppsett

Standard imports.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import sklearn.linear_model as skl
```

Train/test slik foreleser ofte gjør det.

```
np.random.seed(123)
n = len(df)
ntrain = n // 2
ind = np.random.choice(np.arange(n), size=ntrain, replace=False)
mask = np.ones(n, dtype=bool)
mask[ind] = False
train = df.iloc[ind]
test = df.iloc[mask]
```

X er forklaringsvariabler/prediktorer. y er responsen vi forklarer eller predikerer.

```
X = pd.get_dummies(df.drop(columns="y"), drop_first=True)
X = X.astype(float).fillna(0)
y = df["y"]
```

Evalueringsmål.

```
mse = np.mean((y - pred)**2)           # gj.sn. kvadrert feil
mad = np.mean(np.abs(y - pred))         # gj.sn. absoluttfeil
rss = np.sum((y - pred)**2)             # sum kvadrerte feil
tss = np.sum((y - y.mean())**2)         # total variasjon
r2 = 1 - rss / tss
accuracy = np.mean(y_true == pred)
```

Klassifikasjonstabell.

```
pd.crosstab(y_true.values, pred,
            rownames=["Actual"], colnames=["Predicted"])
pd.crosstab(y_true, pred, normalize="index")
```

1. Ordinær lineær regresjon

1. Forklaring

Lineær regresjon modellerer gjennomsnittet til en kontinuerlig respons som en rett linje eller et lineært plan i prediktorene. Den brukes når du vil forklare eller predikere en numerisk y, og når en lineær sammenheng er en rimelig start.

Den gir to nyttige ting på eksamen: prediksjoner og tolkbare koeffisienter. En positiv koeffisient betyr at høyere x henger sammen med høyere forventet y, gitt de andre variablene i modellen.

2. Manuell kode og formel

Formel: $y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + e$

Mål: minimiser summen av kvadrerte residualer: $\sum_i (y_i - \hat{y}_i)^2$

Analytisk løsning: $\hat{\beta} = (X'X)^{-1}X'y$

```
Xc = np.column_stack([np.ones(len(y)), X])
b = np.linalg.solve(Xc.T @ Xc, Xc.T @ y)
pred = Xc @ b
rss = np.sum((y - pred)**2)
tss = np.sum((y - y.mean())**2)
r2 = 1 - rss / tss
```

3. Kode med pakker

Foreleser bruker `statsmodels` når inferens og tabeller er relevante.

```
import statsmodels.api as sm

X_train_const = sm.add_constant(X_train)
ols = sm.OLS(y_train, X_train_const).fit()
print(ols.summary())
```

```
X_test_const = sm.add_constant(X_test)
pred = ols.predict(X_test_const)
```

Med ISLP `ModelSpec`:

```
from ISLP.models import ModelSpec as MS, summarize

design = MS(["horsepower"]).fit(Auto)
X = design.transform(Auto)
y = Auto["mpg"]
res = sm.OLS(y, X).fit()
summarize(res)
```

4. Annet relevant

- P-verdi tester om koeffisienten kan være null, gitt modellen.
- Konfidensintervall handler om forventet respons.
- Prediksjonsintervall handler om en ny observasjon og er bredere.
- Mange variabler og lite data kan gi overfitting; vurder Ridge eller LASSO-regresjon.

2. Ridge-regresjon

1. Forklaring

Ridge er lineær regresjon med straff på store koeffisienter. Den brukes når vanlig lineær regresjon får ustabile koeffisienter, ofte fordi prediktorer er korrelerte eller fordi det er mange prediktorer.

Ridge gjør modellen enklere: koeffisientene krympes mot null, men blir vanligvis ikke nøyaktig null.

2. Manuell kode og formel

Formel: minimer $\sum_i (y_i - x'_i \beta)^2 + \lambda \sum_j \beta_j^2$

Forelesers 2025-eksamen: trekk fra gjennomsnittet i y og i hver X-kolonne, men ikke standardiser.

```
def ridge_manual(X, y, la):
    b0 = y.mean()
    yd = y - b0
    Xd = X - X.mean(axis=0)
    p = Xd.shape[1]
    A = Xd.T @ Xd + la * np.eye(p)
    b1 = np.linalg.solve(A, Xd.T @ yd)
    return b0, b1
```

```
b0, b1 = ridge_manual(X, y, la=1000)
pred = b0 + (X - X.mean(axis=0)) @ b1
```

2025-funksjonen som skal forklares:

```
def f(b1, X, y, la):
    return np.sum((y - X @ b1)**2) + la * np.sum(b1**2)
```

f er summen av kvadrerte residualer pluss Ridge-straff.

3. Kode med pakker

Forelesers `ex611.py`-mønster:

```
import sklearn.linear_model as skl

alphas = np.logspace(-4, 4, 200)
ridge_cv = skl.RidgeCV(alphas=alphas)
ridge_cv.fit(Xtrain, ytrain)

ridgemin = skl.Ridge(alpha=ridge_cv.alpha_)
ridgemin.fit(Xtrain, ytrain)
pred = ridgemin.predict(Xtest)
```

4. Annet relevant

- `la=0` i manuell Ridge gir samme koeffisienter som vanlig lineær regresjon uten intercept.
- Høy `la`: mer bias, lavere varians.
- Lav `la`: mer fleksibel modell, høyere varians.

- Hvis `g()` demeaner `X`, må prediksjoner også bruke demeanet `X`.

Eksakt leave-one-out-mønster for Ridge fra 2025:

```
def loo_ridge(la, X, y):
    n = len(y)
    b0 = y.mean()
    yd = y - b0
    Xd = X - X.mean(axis=0)
    ydpred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
        _, b1 = ridge_manual(X[idx], y[idx], la)
        ydpred[i] = Xd[i] @ b1
    return np.mean((yd - ydpred)**2)

lavec = np.linspace(4500, 5000, 10)
test_mse = np.array([loo_ridge(la, X, y) for la in lavec])
best_la = lavec[np.argmin(test_mse)]
```

3. LASSO-regresjon

1. Forklaring

LASSO står for “least absolute shrinkage and selection operator”. Det er lineær regresjon med straff på absoluttverdien av koeffisientene. Den brukes når du tror at bare noen av prediktorene er viktige.

I motsetning til Ridge kan LASSO-regresjon sette koeffisienter nøyaktig lik null. Derfor fungerer metoden også som variabelutvelgelse.

2. Manuell kode og formel

Formel: minimer $\sum_i (y_i - x_i' \beta)^2 + \lambda \sum_j |\beta_j|$

Det finnes ikke en enkel lukket formel som for vanlig lineær regresjon og Ridge. Konseptuelt brukes soft-thresholding i koordinatvis optimalisering:

`beta_j` blir satt til 0 hvis gevinsten i summen av kvadrerte feil ikke er stor nok til å betale L1-straffen `lambda * abs(beta_j)`

3. Kode med pakker

Forelesers `ex611.py`-mønster:

```
alphas = np.logspace(-4, 4, 200)
lasso_cv = skl.LassoCV(alphas=alphas)
lasso_cv.fit(Xtrain, ytrain)
```

```
lasso = skl.Lasso(alpha=lasso_cv.alpha_)
lasso.fit(Xtrain, ytrain)
pred = lasso.predict(Xtest)
```

Sammenlign koeffisienter:

```
lasso_coef = np.concatenate([lasso.intercept_], lasso.coef_))
```

4. Annet relevant

- Standardisering er ofte viktig fordi straffen avhenger av skalaen på variablene.
- `LassoCV` velger `alpha` med kryssvalidering.
- Mange null-koeffisienter betyr at LASSO-regresjonen har valgt bort variabler.

4. Leave-one-out-kryssvalidering

1. Forklaring

Leave-one-out-kryssvalidering brukes til å velge modellparameter, for eksempel `K` i nearest neighbours eller `la` i Ridge. Metoden trener modellen `n` ganger. Hver gang holdes én observasjon utenfor og predikeres.

2. Manuell kode og formel

Formel for kryssvalideringsfeil: $\frac{1}{n} \sum_i (y_i - \hat{y}_{-i})^2$

```
def loo_score(model_func, X, y):
    n = len(y)
    pred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
```

```

    pred[i] = model_func(X[i], X[idx], y[idx])
return np.mean((y - pred)**2)

```

3. Kode med pakker

Foreleser gjør ofte leave-one-out manuelt i eksamenslignende oppgaver. For vanlig kryssvalidering i sklearn brukes `GridSearchCV` eller modeller som `RidgeCV` og `LassoCV`.

```

from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X_train, y_train)
best = grid.best_estimator_

```

4. Annet relevant

- Brukes til modellvalg, ikke til endelig evaluering hvis du har testsett.
- Leave-one-out er nyttig ved små datasett, men kan være tregt.
- I 2025 måtte man velge 1a for Ridge med leave-one-out.

5. Bootstrap

1. Forklaring

Bootstrap brukes til å simulere samplingsfordelingen til en statistikk uten å samle nye data. Man trekker nye datasett fra det opprinnelige datasettet med tilbakelegging.

2. Manuell kode og formel

For varians: $S^2 = \frac{1}{n-1} \sum_i (y_i - \bar{y})^2$

```

np.random.seed(42)
n = len(y)
B = 10000
vboot = np.zeros(B)

for b in range(B):
    yboot = np.random.choice(y, size=n, replace=True)
    vboot[b] = np.var(yboot, ddof=1)

ci = np.percentile(vboot, [2.5, 97.5])
plt.hist(vboot, bins=40)

```

3. Kode med pakker

Foreleser gjør bootstrap direkte med `np.random.choice`; ingen egen pakke er nødvendig.

4. Annet relevant

- `replace=True` er hele poenget.
- `ddof=1` gir stikkprøvevariens, altså deling på `n-1`.
- 95 prosent konfidensintervall med persentilmetoden: 2.5 og 97.5 prosentil.

6. Logistisk regresjon

1. Forklaring

Logistisk regresjon brukes når responsen er binær, for eksempel 0/1, Up/Down eller `claim/no claim`. Modellen predikerer sannsynligheten for klasse 1.

Koeffisientene virker lineært på log-odds, ikke direkte på sannsynligheten.

2. Manuell kode og formel

Formel: $P(Y = 1|X = x) = \frac{1}{1 + \exp[-(\beta_0 + \beta'x)]}$

```

eta = b0 + X @ b
prob = 1 / (1 + np.exp(-eta))
pred = (prob > 0.5).astype(int)

```

Krysstabell:

```

pd.crosstab(y_true, pred, rownames=["Actual"], colnames=["Predicted"])

```

3. Kode med pakker

Foreleser bruker `statsmodels.formula.api` i Weekly-oppgaven:

```
import statsmodels.formula.api as smf
from ISLP.models import summarize

Weekly["y"] = (Weekly["Direction"] == "Up").astype(int)
m = smf.logit("y ~ Volume + Lag1 + Lag2 + Lag3 + Lag4 + Lag5",
             data=train).fit()
summarize(m)
prob = m.predict(test)
pred = prob > 0.5
```

Statsmodels-variant uten formel:

```
X_train = sm.add_constant(train[["Lag2"]])
y_train = (train["Direction"] == "Up").astype(int)
m = sm.GLM(y_train, X_train, family=sm.families.Binomial()).fit()
```

4. Annet relevant

- Ved ubalanserte klasser kan terskel 0.5 være dårlig.
- Finn terskel på treningsdata, evaluer på testdata.
- Bruk rad-proporsjoner i krysstabell for å se hvordan modellen gjør det per klasse.

7. K-nearest neighbours og lokal lineær regresjon

1. Forklaring

K-nearest neighbours predikerer med de K nærmeste observasjonene. Ved regresjon brukes gjennomsnittet av responsen. Ved klassifikasjon brukes andelen i klasse 1.

2024-eksamen brukte en variant: velg de K nærmeste, men kjør lineær regresjon lokalt på dem før prediksjon.

2. Manuell kode og formel

Vanlig nearest neighbours-regresjon:

Formel: $\hat{f}(x_0) = \frac{1}{K} \sum_{i \in N_K(x_0)} y_i$

```
def knn(x0, x, y, K=3):
    d = np.abs(x - x0)
    idx = np.argsort(d)[:K]
    return np.mean(y[idx])
```

Lokal lineær regresjon fra 2024-eksamen:

```
def local_lm(x0, x, y, K=3):
    d = np.abs(x - x0)
    o = np.argsort(d)[:K]
    x1 = x[o]
    y1 = y[o]
    X1 = sm.add_constant(x1)
    reg = sm.OLS(y1, X1).fit()
    return reg.predict([1, x0])[0]
```

3. Kode med pakker

```
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
prob = knn.predict_proba(X_test)[: , 1]
pred = prob > 0.5
```

4. Annet relevant

- Lav K: mer fleksibel, mer varians, kan overfitte.
- Høy K: glattere, mer bias, kan underfitte.
- Med flere prediktorer bør X standardiseres.
- Flere prediktorer: bruk euklidsk avstand, `np.sqrt(np.sum((X-x0)**2, axis=1))`.

8. Linear discriminant analysis, quadratic discriminant analysis og Naive Bayes

1. Forklaring

Dette er klassifikasjonsmodeller som estimerer sannsynligheten for klasse gitt prediktorene.

- Linear discriminant analysis antar omtrent normalfordelte prediktorer og lik kovarians i klassene.
- Quadratic discriminant analysis tillater ulik kovarians i klassene og er mer fleksibel.
- Naive Bayes antar at prediktorene er uavhengige gitt klassen.

2. Manuell kode og formel

Bayes-idé: $P(Y = k|X = x) \propto P(X = x|Y = k)P(Y = k)$

For linear discriminant analysis med to klasser brukes klassegjennomsnitt og felles kovariansmatrise:

```
mu0 = X[y == 0].mean(axis=0)
mu1 = X[y == 1].mean(axis=0)
pi0 = np.mean(y == 0)
pi1 = np.mean(y == 1)
```

3. Kode med pakker

Forelesers Weekly-mønster:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
```

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
prob = lda.predict_proba(X_test)
pred = np.where(prob[:, 1] > 0.54, "Up", "Down")
```

```
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
```

4. Annet relevant

- Linear discriminant analysis er ofte mer stabil ved lite data.
- Quadratic discriminant analysis er mer fleksibel, men trenger mer data.
- Foreleser justerer terskelen i Weekly-eksempelet, for eksempel 0.54 og 0.55.

9. Polynomregresjon og trappefunksjoner

1. Forklaring

Polynomregresjon lar en lineær modell fange kurvede sammenhenger ved å legge inn x , x^2 , x^3 osv. Trappefunksjoner deler en kontinuerlig variabel inn i intervaller.

2. Manuell kode og formel

Formel: $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_K x^K + e$

```
K = 7
Xpoly = np.vander(age, K + 1, increasing=True)
fit = sm.OLS(wage, Xpoly).fit()
```

Velg grad med validation set:

```
mse = []
for k in range(1, 11):
    Xtr = np.vander(age_train, k + 1, increasing=True)
    fit = sm.OLS(wage_train, Xtr).fit()
    Xte = np.vander(age_test, k + 1, increasing=True)
    mse.append(np.mean((wage_test - fit.predict(Xte))**2))
best_k = np.argmin(mse) + 1
```

3. Kode med pakker

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=3, include_bias=True)
Xpoly = poly.fit_transform(X_train)
```

4. Annet relevant

- Høy grad kan overfitte.

- Step function med `pd.cut`:

```
train = train.assign(age_cut=pd.cut(train["age"], 4))
Xtr = pd.get_dummies(train["age_cut"], drop_first=True).astype(float)
Xtr = sm.add_constant(Xtr)
```

10. Regresjonssplines

1. Forklaring

Splines modellerer ikke-lineære sammenhenger ved å lime sammen polynomer. De er ofte mer stabile enn høye polynomgrader.

2. Manuell kode og formel

Kubisk spline med knutepunkt `xi` bruker funksjonen:

Formel: $h(x, \xi) = \max((x - \xi)^3, 0)$

```
def h(x, xi):
    return np.maximum((x - xi)**3, 0)
```

```
X_sp = np.column_stack([np.ones(n), x, x**2, x**3,
                        h(x, 3.0), h(x, 6.0)])
fit = sm.OLS(y, X_sp).fit()
```

3. Kode med pakker

Foreleser bruker `BSpline` fra ISLP:

```
from ISLP.transforms import BSpline
```

```
bs = BSpline(internal_knots=[3.5, 5.5], degree=3, intercept=True)
Xbs = bs.fit_transform(x.reshape(-1, 1))
fit = sm.OLS(y, Xbs).fit()
```

4. Annet relevant

- Kubisk spline med K interne knutepunkter har $K + 4$ basisfunksjoner.
- Natural spline er lineær i halene.
- Velg knutepunkter med leave-one-out eller ved å teste rimelige kombinasjoner.

11. Generalized additive model

1. Forklaring

En generalized additive model lar hver prediktor ha sin egen funksjon:

Formel: $y = b_0 + f_1(x_1) + f_2(x_2) + \dots + e$

Den brukes når noen sammenhenger er ikke-lineære, men du fortsatt vil tolke variablene separat.

2. Manuell kode og formel

Foreleser viser backfitting med nearest neighbours-smoother:

```
def knn(x0, x, y, K=3):
    d = np.abs(x - x0)
    idx = np.argsort(d)[:K]
    return np.mean(y[idx])

b0 = y.mean()
yd = y - b0
K = 20
f1 = b0 + np.array([knn(x0, x1, yd, K) for x0 in x1])
res = y - f1
res = res - res.mean()
f2 = b0 + np.array([knn(x0, x2, res, K) for x0 in x2])
```

Backfitting gjentar dette: estimer én funksjon om gangen på residualen fra de andre.

3. Kode med pakker

Python-variant med B-splines og `statsmodels`:

```
from ISLP.transforms import BSpline
```

```
bs = BSpline(internal_knots=[-1, 0, 1], degree=3, intercept=False)
X3_sp = bs.fit_transform(X[:, 2].reshape(-1, 1))
X_gam = np.column_stack([np.ones(len(y)), X[:, 0], X[:, 1],
                        X3_sp, X[:, 3:]])
```

```
gam = sm.OLS(y, X_gam).fit()
```

4. Annet relevant

Finn ikke-linearitet ved å sammenligne lineær og kvadratisk modell:

```
r2_lin = sm.OLS(y, sm.add_constant(xi)).fit().rsquared
Xq = sm.add_constant(np.column_stack([xi, xi**2]))
r2_quad = sm.OLS(y, Xq).fit().rsquared
delta = r2_quad - r2_lin
```

I 2025-eksamen var poenget å vise om generalized additive model forbedrer treningsfeil og forklart variasjon.

12. Beslutningstre

1. Forklaring

Et beslutningstre deler data i grupper med binære splitt. For regresjon predikerer treet gjennomsnittet i bladet. For klassifikasjon predikerer treet majoritetsklassen eller sannsynligheten i bladet.

2. Manuell kode og formel

Regresjonstre velger split som minimerer summen av residualfeil i de to delene:

Formel: summen av kvadrerte residualer i region 1 pluss summen av kvadrerte residualer i region 2.

```
from scipy.optimize import minimize_scalar

def RSS_split(s, x, y):
    R1 = y[x < s]
    R2 = y[x >= s]
    if len(R1) == 0 or len(R2) == 0:
        return np.inf
    return np.sum((R1 - R1.mean())**2) + np.sum((R2 - R2.mean())**2)

res = minimize_scalar(RSS_split, bounds=(x.min(), x.max()),
                      args=(x, y), method="bounded")
```

3. Kode med pakker

```
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier, plot_tree

tree = DecisionTreeRegressor(min_samples_leaf=15, max_depth=3,
                             random_state=0)

tree.fit(X_train, y_train)
plot_tree(tree, feature_names=X_train.columns, filled=False, fontsize=8)
pred = tree.predict(X_test)
```

Pruning:

```
from sklearn.model_selection import GridSearchCV

path = tree.cost_complexity_pruning_path(X_train, y_train)
grid = GridSearchCV(DecisionTreeRegressor(random_state=0),
                    {"ccp_alpha": path.ccp_alphas}, cv=5)
grid.fit(X_train, y_train)
best_tree = grid.best_estimator_
```

4. Annet relevant

- Ubegrensede trær overfitter lett.
- `min_samples_leaf` og `max_depth` gjør treet enklere.
- Trær er lette å tolke, men kan være ustabile.

13. Bagging, random forest og boosting

1. Forklaring

Bagging trener mange trær på bootstrap-utvalg og tar gjennomsnitt/flertall. Random forest gjør det samme, men vurderer bare et tilfeldig utvalg prediktorer ved hvert split. Boosting bygger trær sekvensielt, der nye trær prøver å rette opp feil fra tidligere trær.

2. Manuell kode og formel

Bagging-formel for regresjon: $\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$

```
from sklearn.tree import DecisionTreeRegressor

preds = []
for b in range(B):
    idx = np.random.choice(len(X_train), len(X_train), replace=True)
    tree_b = DecisionTreeRegressor(random_state=b)
    tree_b.fit(X_train.iloc[idx], y_train.iloc[idx])
    preds.append(tree_b.predict(X_test))
pred_bag = np.mean(preds, axis=0)
```

3. Kode med pakker

Forelesers `ames.py`-mønster:

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

bagging = RandomForestRegressor(n_estimators=500,
                               max_features=X_train.shape[1], random_state=123)
bagging.fit(X_train, y_train)

rf = RandomForestRegressor(n_estimators=500,
                           max_features=3, random_state=123)
rf.fit(X_train, y_train)

boost = GradientBoostingRegressor(n_estimators=1000,
                                   learning_rate=0.01, max_depth=2, random_state=123)
boost.fit(X_train, y_train)
```

Klassifikasjon:

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

rfc = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                             random_state=123)
rfc.fit(X_train, y_train)
prob = rfc.predict_proba(X_test)[: , 1]
pred = prob > cutoff
```

4. Annet relevant

- Bagging: `max_features` = antall prediktorer.
- Random forest: lavere `max_features`, for eksempel 3 eller "sqrt".
- Variabelviktighet:

```
pd.Series(rf.feature_importances_,
          index=X_train.columns).sort_values().plot.barh()
```

14. Support vector machine

1. Forklaring

Support vector machine er en klassifikasjonsmetode som forsøker å lage en god beslutningsgrense mellom klassene. Med kernel kan grensen bli ikke-lineær.

2. Manuell kode og formel

Intuisjon: finn et hyperplan som skiller klassene med størst mulig margin, men tillat noen feil når data ikke kan skilles perfekt. For radial basis-kernel:

Formel: $K(x, x') = \exp(-\gamma ||x - x'||^2)$

Ingen manuell implementasjon forventes i faget.

3. Kode med pakker

Forelesers `ex97.py`-mønster:

```

from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

param_rbf = {"C": [0.01, 0.1, 1, 10, 100, 1000],
             "gamma": [0.5, 1, 2, 3, 4]}
tuneCg = GridSearchCV(SVC(kernel="rbf"), param_rbf, cv=5)
tuneCg.fit(X, y)
best = tuneCg.best_estimator_
pred = best.predict(X_test)

```

4. Annet relevant

- I sklearn betyr liten C sterkere regulering og mykere margin.
- Stor gamma gir mer lokal og kompleks grense.
- Foreleser bruker også `kernel="linear"` og `kernel="poly"`.

15. Nevrale nettverk

1. Forklaring

Nevrale nettverk i dette faget betyr flerlags perceptron fra sklearn. Det brukes som fleksibel prediksjonsmodell. Foreleser har presisert at PyTorch, convolutional neural networks og recurrent neural networks ikke er pensum her.

2. Manuell kode og formel

For ett skjult lag:

Formel: $h = \text{ReLU}(XW_1 + b_1)$ og $\hat{y} = hW_2 + b_2$

```

def relu(z):
    return np.maximum(0, z)

```

```

h = relu(X @ W1 + b1)
pred = h @ W2 + b2

```

Manuell trening forventes ikke.

3. Kode med pakker

Forelesers `ames.py`-mønster:

```

from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_stand = scaler.fit_transform(X_train)
X_test_stand = scaler.transform(X_test)

nn = MLPRegressor(hidden_layer_sizes=(15,),
                  activation="relu",
                  max_iter=100000,
                  random_state=0)
nn.fit(X_train_stand, y_train)
pred = nn.predict(X_test_stand)

```

4. Annet relevant

- Standardisering er viktig.
- (15,) betyr ett skjult lag med 15 noder.
- (15, 8) betyr to skjulte lag.
- Resultater kan variere; bruk `random_state`.

16. Principal component analysis / hovedkomponentanalyse

1. Forklaring

Principal component analysis, på norsk hovedkomponentanalyse, brukes når vi bare har forklaringsvariabler og ønsker å oppsummere mange variabler i færre nye variabler. Første hovedkomponent er den lineære kombinasjonen som forklarer mest varians.

2. Manuell kode og formel

Formel: $Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p$

Velg vektene slik at $\text{Var}(Z_1)$ blir størst, med krav om at vektene har kvadratsum 1.

Scores kan beregnes slik når `components_` er funnet:

```
scores = X_scaled @ pca.components_.T
```

3. Kode med pakker

Foreleser bruker Python-klassen PCA og StandardScaler:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
def summary_pca(fit):
    return pd.DataFrame({
        "Standard deviation": np.sqrt(fit.explained_variance_),
        "Proportion of Variance": fit.explained_variance_ratio_,
        "Cumulative Proportion": np.cumsum(fit.explained_variance_ratio_)
    })
```

```
X_scaled = StandardScaler(with_mean=True, with_std=True).fit_transform(X)
pca = PCA()
pca.fit(X_scaled)
summary_pca(pca)
```

Loadings:

```
loadings = pd.DataFrame(pca.components_.T, index=X.columns,
                        columns=[f"PC{i+1}" for i in range(pca.n_components_)])
```

4. Annet relevant

- Standardiser hvis variablene har ulik skala.
- Loadings sier hvordan originalvariablene inngår i komponentene.
- Scores sier hvor observasjonene ligger på komponentene.
- Brukes til utforskende/deskriptiv analyse, ikke prediksjon av y.

17. Clustering

1. Forklaring

Clustering grupperer observasjoner uten at vi har en responsvariabel. K-means krever at du velger antall grupper på forhånd. Hierarkisk clustering bygger et dendrogram som kan kuttes på ulike nivåer.

2. Manuell kode og formel

K-means minimerer within-cluster variation:

Formel: $\sum_k \sum_{i \in C_k} \|x_i - \mu_k\|^2$

```
def kmeans_simple(X, K, iters=100):
    idx = np.random.choice(len(X), K, replace=False)
    centers = X[idx].copy()
    for _ in range(iters):
        dists = np.array([np.sum((X - c)**2, axis=1) for c in centers])
        labels = np.argmin(dists, axis=0)
        centers = np.array([X[labels == k].mean(axis=0) for k in range(K)])
    return labels, centers
```

3. Kode med pakker

Foreleser bruker disse pakkene i forelesning 14:

```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, cut_tree
from ISLP.cluster import compute_linkage
```

```
X_s = StandardScaler().fit_transform(X)

km = KMeans(n_clusters=3, random_state=1, n_init=10)
km.fit(X_s)

hc = AgglomerativeClustering(distance_threshold=0, n_clusters=None,
                             linkage="complete")
hc.fit(X_s)
lm = compute_linkage(hc)
dendrogram(lm, no_labels=True)
clusters = cut_tree(lm, n_clusters=3).reshape(-1)
```

4. Annet relevant

- Standardiser før avstandsbaserte metoder.
- Complete linkage bruker maks avstand mellom grupper.
- Single linkage bruker minste avstand.
- Average linkage bruker gjennomsnittlig avstand.
- Clustering er hypotesegenererende og deskriptivt.

18. Caseoppgaver på eksamen

1. Forklaring

Caseoppgaver handler ofte om enten forklaring eller prediksjon. Før du modellerer må du vite hvilket spørsmål du svarer på.

2. Manuell/logisk fremgangsmåte

Forklaring: “Hvorfor skjer Y?”

1. Gjør deskriptiv analyse.
2. Bruk logistisk regresjon eller lineær regresjon for tolkbare koeffisienter.
3. Bruk trær/random forest for ikke-linearitet og variabelviktighet.
4. Svar med de viktigste variablene.

Prediksjon: “Hvem får Y i fremtiden?”

1. Fjern lekkasjevariabler.
2. Fjern variabler som ikke er kjent på prediksjonstidspunktet.
3. Del i treningsdata og testdata.
4. Velg terskel på treningsdata hvis klassene er ubalanserte.
5. Evaluer på testdata.

3. Kode med pakker

Typisk evaluering:

```
# statsmodels-formelmodell:
prob = model.predict(test)
pred = prob > cutoff
pd.crosstab(test["y"], pred, normalize="index")
np.mean(test["y"] == pred)

# sklearn-klassifikasjonsmodell:
prob = model.predict_proba(X_test)[: , 1]
pred = prob > cutoff
pd.crosstab(y_test, pred, normalize="index")
```

4. Annet relevant

- “Random forest forbedrer prediksjon, men logistisk regresjon er lettere å tolke.”
- “Variabelviktighet viser prediktiv betydning, ikke nødvendigvis kausal effekt.”
- “Jeg bruker testsettet bare til siste evaluering.”
- “Terskel 0.5 er ikke nødvendigvis passende hvis positiv klasse er sjelden.”